



Assignment of “Software Engineering & Design Pattern”

Course Name: Software Engineering & Design Pattern

Submitted to:

Md. Mushtaq Shahriyar Rafee

Assistant Professor and Head Dept. of Data Science

Metropolitan University, Sylhet.

Submitted By:

Name: Jual Ahmed

ID: 241-112-006

Batch: 34th (EVE)

Department of Computer Science & Engineering

Metropolitan University, Sylhet.

Date of Submission: 20/04/2026

Software Requirements Specification (SRS)

For Hotel Room Booking System

Version 1.0 approved

Prepared by
Jual Ahmed (241-112-006)

Department of Computer Science & Engineering
Metropolitan University, Sylhet

Date Created: 20-04-2026

Revision History

V	D	A	D
e	a	u	e
r	t	t	s
s	e	h	c
i		o	r
o		r	i
n		(	p
		s	t
		)	i
			o
			n
1	2	A	I
.	0	z	n
0	-	m	i

0	o	t
4	l	i
-	H	a
2	o	l
0	s	S
2	s	R
6	a	S
	n	
	,	
	J	
	u	
	a	
	l	
	A	
	h	
	m	
	e	
	d	

Table of Contents

- 1. Introduction
 - 1.1 Purpose
 - 1.2 Document Conventions
 - 1.3 Intended Audience and Reading Suggestions
 - 1.4 Project Scope
 - 1.5 References
- 2. Overall Description
 - 2.1 Product Perspective
 - 2.2 Product Features
 - 2.3 User Classes and Characteristics
 - 2.4 Operating Environment
 - 2.5 Design and Implementation Constraints

- 2.6 User Documentation
- 2.7 Assumptions and Dependencies
- 3. System Features
 - 3.1 User Registration and Login
 - 3.2 View Available Rooms
 - 3.3 Online Room Booking
 - 3.4 Booking History
 - 3.5 Admin Dashboard
 - 3.6 Room Management
- 4. External Interface Requirements
 - 4.1 User Interfaces
 - 4.2 Hardware Interfaces
 - 4.3 Software Interfaces
 - 4.4 Communications Interfaces
- 5. Other Nonfunctional Requirements
 - 5.1 Performance Requirements
 - 5.2 Safety Requirements
 - 5.3 Security Requirements
 - 5.4 Software Quality Attributes
- 6. Other Requirements

Appendix A: Analysis Models

1. Introduction

1.1 Purpose

The purpose of this document is to specify the software requirements for the Hotel Room Booking System. This system is a web-based application that allows customers to browse available hotel rooms and book them online. It also provides hotel administrators with tools to manage room inventory, view booking records, and maintain customer information. This SRS serves as a complete guide for the development team to design, implement, and test the system.

1.2 Document Conventions

- Bold text indicates key terms or important concepts.

- REQ-<number> labels uniquely identify each functional requirement.
- Priority levels: High (essential for first release), Medium (important but can be delayed), Low (nice to have).
- All requirements are written with “The system shall ...” or “The user shall be able to ...”.

1.3 Intended Audience and Reading Suggestions

This document is intended for:

- Developers – to understand functional and non-functional requirements.
- Project managers – to track scope and plan iterations.
- Testers – to derive test cases.
- Instructor (Abu Jafar Md Jakarta) – to evaluate project completeness.

Readers should start with Section 1 (Introduction) and Section 2 (Overall Description). Section 3 contains detailed functional requirements. Non-functional requirements are in Section 5.

1.4 Project Scope

The Hotel Room Booking System is a web-based application that enables:

- Customer account creation and login.
- Browsing of available rooms with details.
- Online booking of rooms (without integrated payment in the first version).
- Viewing personal booking history.
- Admin management of rooms (add, update, delete) and booking records.

The first version will not include an online payment gateway, mobile application support, or third-party hotel service integration. Those are considered future enhancements.

1.5 References

- Project Proposal: 241-112-004_241-112-006_ProjectProposal_300.pdf
 - MERN Stack Documentation: <https://www.mongodb.com/mern-stack>
 - React (Next.js) Documentation: <https://nextjs.org/docs>
 - Node.js/Express Documentation: <https://expressjs.com>
 - MongoDB Documentation: <https://docs.mongodb.com>
-

2. Overall Description

2.1 Product Perspective

The Hotel Room Booking System is a new, self-contained web application. It replaces manual or paper-based hotel booking processes. The system consists of two main sub-systems:

- User Panel – for customers to register, browse, book, and view history.
- Admin Panel – for hotel staff to manage rooms and bookings.

The system will be built using the MERN stack (MongoDB, Express.js, React with Next.js, Node.js). It will be deployed on Vercel for public access. No direct integration with external hotel systems is required initially.

2.2 Product Features

- User registration and login – secure account creation and authentication.
- View available rooms – display rooms with details (type, price, capacity, availability).
- Online room booking – select dates and confirm a booking.
- Booking history – each user can see past and upcoming bookings.
- Admin dashboard – overview of all bookings and rooms.
- Room management – admin can add, edit, or delete room entries.

2.3 User Classes and Characteristics

U	Description	Privileges
s		
e		
r		
C		
l		
a		
s		
s		

C u s t o m e r (G u e s t)	End user who wants to book a hotel room. Can register, log in, view rooms, book, and view history.	Low – only own data.
A d m i n i s t r a t o r	Hotel staff member responsible for managing rooms and overseeing bookings. Has full access to admin panel.	High – can modify any room or booking.

2.4 Operating Environment

- Server-side: Node.js runtime, Express.js framework, MongoDB database.
- Client-side: Modern web browsers (Chrome, Firefox, Edge, Safari) on desktop, tablet, or mobile devices.
- Deployment: Vercel (frontend), MongoDB Atlas (database), and a cloud service (e.g., Render or Cyclic) for backend API if needed.
- Development OS: Windows/Linux/macOS.

2.5 Design and Implementation Constraints

- Technology stack is fixed to MERN (MongoDB, Express, React/Next.js, Node.js).
- Frontend framework must be Next.js (React-based).
- Version control must be Git with GitHub repository.
- No payment gateway integration in the first release.
- The system must be responsive and work on different screen sizes.
- All passwords must be hashed (using bcrypt) before storing.

2.6 User Documentation

The following user documents will be delivered:

- User manual (PDF) explaining how to register, log in, book a room, and view history.
- Admin manual (PDF) describing how to manage rooms and bookings.
- Online help – simple tooltips and a “Help” page within the application.

2.7 Assumptions and Dependencies

- Users have basic internet access and a modern web browser.
 - The hotel has a stable internet connection for the admin panel.
 - The development team has access to required software (Node.js, MongoDB, Git).
 - The project depends on the availability of free tiers of MongoDB Atlas and Vercel for deployment.
 - No third-party API (e.g., payment gateway) is required for the first version.
-

3. System Features

3.1 User Registration and Login (Priority: High)

3.1.1 Description and Priority

New customers can create an account using their email address and a password. Registered users can log in to access booking features. Priority: High – essential for user identification.

3.1.2 Stimulus/Response Sequences

1. User clicks “Register” → enters name, email, password → system validates input → creates account → redirects to login.
2. User enters email/password → system verifies credentials → if valid, user is logged in and redirected to home page.
3. If invalid credentials → error message is shown.

3.1.3 Functional Requirements

- REQ-1.1 – The system shall allow a new user to register with a unique email address, full name, and password (minimum 6 characters).
- REQ-1.2 – The system shall encrypt passwords using bcrypt before storing them in the database.
- REQ-1.3 – The system shall require users to log in before accessing booking or history pages.
- REQ-1.4 – The system shall provide a “Logout” option that terminates the session.

3.2 View Available Rooms (Priority: High)

3.2.1 Description and Priority

Customers can see a list of all hotel rooms that are currently available (not booked for selected dates). Priority: High – core feature.

3.2.2 Stimulus/Response Sequences

1. User navigates to “Rooms” page → system fetches all rooms from database → displays each room with type, price, capacity, and an image placeholder.
2. User optionally selects check-in/check-out dates → system shows only rooms that are free during that period.

3.2.3 Functional Requirements

- REQ-2.1 – The system shall display a list of rooms with at least: room number, type (e.g., single, double, suite), price per night, and maximum occupancy.
- REQ-2.2 – The system shall allow filtering by date range (check-in, check-out) to show only available rooms.
- REQ-2.3 – The system shall indicate if a room is currently booked for the selected dates.

3.3 Online Room Booking (Priority: High)

3.3.1 Description and Priority

A logged-in user can select a room, choose dates, and place a booking. Priority: High.

3.3.2 Stimulus/Response Sequences

1. User clicks “Book Now” on a room → enters check-in/check-out dates → system checks availability → if free, creates a booking record → shows confirmation message.
2. If dates conflict → system displays “Room not available for selected dates”.

3.3.3 Functional Requirements

- REQ-3.1 – The system shall create a booking record containing: user ID, room ID, check-in date, check-out date, booking date, and status (confirmed/cancelled).
- REQ-3.2 – The system shall prevent double-booking of the same room for overlapping dates.
- REQ-3.3 – The system shall send a confirmation message (on-screen) after successful booking.
- REQ-3.4 – The system shall allow the user to cancel a booking (only if check-in date is in the future).

3.4 Booking History (Priority: Medium)

3.4.1 Description and Priority

Users can view all their past and upcoming bookings. Priority: Medium.

3.4.2 Stimulus/Response Sequences

User clicks “My Bookings” → system retrieves all bookings for that user → displays list with room details, dates, and status.

3.4.3 Functional Requirements

- REQ-4.1 – The system shall display each user’s booking history, sorted by booking date descending.
- REQ-4.2 – Each history entry shall show: room type, check-in/out dates, total price, and status (confirmed/cancelled).
- REQ-4.3 – The user shall be able to cancel a future booking directly from the history page.

3.5 Admin Dashboard (Priority: High)

3.5.1 Description and Priority

An administrator (with special login) can view an overview of all rooms and bookings. Priority: High.

3.5.2 Stimulus/Response Sequences

Admin logs in using admin credentials → redirected to dashboard → sees summary (total rooms, total bookings, recent bookings). Admin can click to manage rooms or view all bookings.

3.5.3 Functional Requirements

- REQ-5.1 – The system shall provide a separate login path for administrators.
- REQ-5.2 – The admin dashboard shall display key metrics: number of rooms, number of active bookings, and a list of recent bookings.
- REQ-5.3 – Only authenticated admins can access the dashboard and management features.

3.6 Room Management (Priority: High)

3.6.1 Description and Priority

Admin can add new rooms, edit existing room details, and delete rooms (if no future bookings exist). Priority: High.

3.6.2 Stimulus/Response Sequences

- Admin goes to “Manage Rooms” → sees list of all rooms → can click “Add Room” → fills form (type, price, capacity, description) → system saves room.
- Admin edits a room → updates fields → saves changes.
- Admin deletes a room → system checks for future bookings → if none, deletes.

3.6.3 Functional Requirements

- REQ-6.1 – The system shall allow admin to add a room with fields: room number, type, price per night, capacity, description, and image URL.
 - REQ-6.2 – The system shall allow admin to edit any existing room details.
 - REQ-6.3 – The system shall allow admin to delete a room only if there are no future bookings for that room.
 - REQ-6.4 – The system shall list all bookings in a table with filters (by room, by date) for admin.
-

4. External Interface Requirements

4.1 User Interfaces

- Customer pages: Home, Rooms (grid view), Room details, Booking form, My Bookings, Login, Register.
- Admin pages: Login, Dashboard, Manage Rooms (list + add/edit form), Manage Bookings (list).
- Design style: Clean, responsive, using HTML5/CSS3, and Tailwind CSS (optional). Buttons and forms with clear labels.
- Error messages: Displayed in red near the relevant field.
- Success messages: Green pop-up or banner.

4.2 Hardware Interfaces

- No direct hardware interfaces. The system runs on standard web servers and client devices with a web browser.

4.3 Software Interfaces

- Database: MongoDB – stores users, rooms, bookings. Communication via Mongoose ODM.
- Backend API: Express.js provides RESTful endpoints (e.g., `/api/users/register`, `/api/rooms`, `/api/bookings`). Frontend consumes these APIs.
- Authentication: JSON Web Tokens (JWT) stored in HTTP-only cookies or localStorage.
- Version control: Git, hosted on GitHub.

4.4 Communications Interfaces

- Protocol: HTTP/HTTPS. All communication between client and server uses HTTPS in production.
 - API format: JSON.
 - Browser-server: Fetch API or Axios.
 - Deployment: Frontend on Vercel, Backend on a Node.js hosting service (e.g., Render), database on MongoDB Atlas.
-

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- Page load time: Home page should load within 2 seconds on a broadband connection.
- Concurrent users: The system should support at least 50 simultaneous users without noticeable delay.
- Booking response: Booking request should be processed within 1 second.

5.2 Safety Requirements

- No critical safety issues. However, the system should prevent data loss by using database backups (MongoDB Atlas automatic backups).

5.3 Security Requirements

- Authentication: Passwords must be hashed (bcrypt, salt rounds ≥ 10).
- Session management: JWT tokens expire after 24 hours.
- Authorization: Admin routes must be protected; regular users cannot access admin endpoints.
- Data validation: All inputs must be sanitized to prevent NoSQL injection and XSS attacks.
- HTTPS: Enforced in production.

5.4 Software Quality Attributes

- Usability: Interface should be intuitive, with clear labels and navigation.
- Reliability: System uptime target of 99% (excluding planned maintenance).
- Maintainability: Code should follow modular architecture (MVC pattern) and be commented.

- Testability: Each API endpoint and UI component should be testable using Postman and React Testing Library.
 - Availability: The system should be accessible 24/7 (subject to hosting provider uptime).
-

6. Other Requirements

- Database requirements: MongoDB collections for `users`, `rooms`, `bookings`. Indexes on `email` (unique) and `roomId` with `date` for availability queries.
 - Internationalization: Only English language in the first version.
 - Legal: None.
 - Reuse: The team may reuse open-source components (e.g., date picker library) under MIT license.
-

Appendix A: Analysis Models

Entity-Relationship Diagram (ERD)

text

```
User (id, name, email, password, role)
```

```
Room (id, roomNumber, type, pricePerNight, capacity, description,  
imageUrl)
```

```
Booking (id, userId, roomId, checkInDate, checkOutDate, totalPrice, status, createdAt)
```

Preliminary Use Case Diagram

Actors: Customer, Administrator

Use Cases:

- Customer: Register, Login, View Rooms, Book Room, Cancel Booking, View History.
- Administrator: Login, Manage Rooms (Add/Edit/Delete), View All Bookings.